



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA
Escuela Profesional de Ingeniería de Sistemas

INFORME DE LABORATORIO
“Snake”

Curso: Sistemas de Información

Docente: Ing. Breno Luque Zuñiga

Peralta Sebán, Daniel Sebastián (2024016803)

Tacna – Perú
2026



INDICE

I.	INFORMACIÓN GENERAL	3
-	Objetivos	3
-	Equipos, materiales, programas y recursos utilizados	3
II.	MARCO TEORICO	3
III.	PROCEDIMIENTO	4
	Programa: Catálogo de Productos y Buscador:.....	4
	1. Diseño del Programa.....	4
	2. Tabla de Controles.....	5
	3. Código del Programa.....	5
IV.	ANALISIS E INTERPRETACION DE RESULTADOS	16
	CONCLUSIONES	16
	RECOMENDACIONES	16
	WEBGRAFIA	17



INFORME DE LABORATORIO

TEMA: Snake

I. INFORMACIÓN GENERAL

- **Objetivos:**
 - Desarrollar un videojuego Snake utilizando C# y Windows Forms.
 - Llevar a cabo programación orientada a objetos utilizando clases y enumeraciones.
 - Controlar el movimiento en tiempo real a partir de Timer.
 - Utilizar los eventos de teclado para controlar la serpiente.
 - Implementar la detección de colisiones.
 - Generar comida aleatoria no determinada en el tablero.
 - Controlar el crecimiento de la serpiente.
 - Gestionar el puntaje y el reinicio de la partida.
 - Implementar el dibujo gráfico a través de Graphics.
- **Equipos, materiales, programas y recursos utilizados:**
 - Computadora con sistema operativo Windows 8
 - Visual Studio
 - Lenguaje de programación C#.
 - .NET Framework

II. MARCO TEORICO

Juego Snake: Snake es un clásico videojuego en el que el jugador consigue el control de una serpiente o culebra que tiene que comer unos objetos para ir aumentando de tamaño evitando que la culebra se choque con los bordes o consigo misma.

Windows Forms: Windows Forms constituirá una biblioteca gráfica de .NET para realizar aplicaciones gráficas de escritorio

Programación Orientada a Objetos: La programación orientada a objetos organiza el sistema por medio de clases y objetos.

En este proyecto se emplean las clases:

- Serpiente
- Comida
- Posicion



Eventos: Los eventos ponen en marcha unas acciones que hacen el usuario o el sistema.

Ejemplos:

- Presionar teclas.
- Pulsar botones.
- Ejecutar temporizadores.

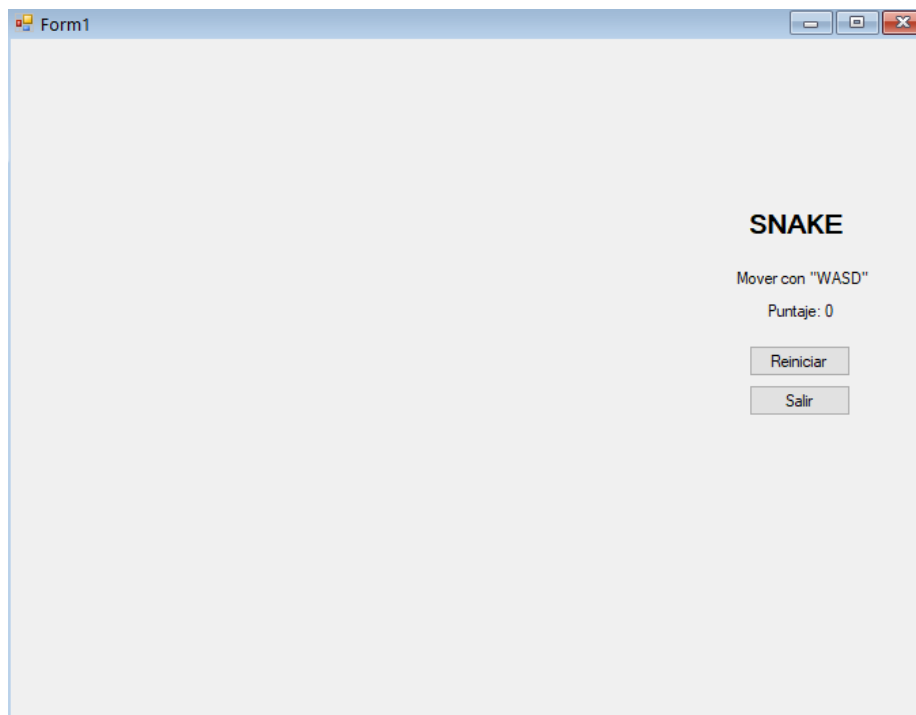
Timer: Timer sirve a ejecutar instrucciones circularmente en tiempos prefijados lo que simula el movimiento del juego.

Graphics: La clase Graphics dibuja figuras geométricas en el formulario.

III. PROCEDIMIENTO

Programa: Catálogo de Productos y Buscador:

1. Diseño del Programa





2. Tabla de Controles

Control	Propiedad	Valor
Label1	Text	SNAKE
	Font	bold type
Label2	Text	Mover con "WASD"
Label3	Text	Puntaje: 0
Label4	Name	lblPuntaje
Button1	Name	btnReiniciar
	Text	Reiniciar
Button2	Name	btnSalir
	Text	Salir
Timer1	Name	timerJuego

3. Código del Programa

Funcionalidad 1: Variables principales

Representa el objeto principal del juego: la serpiente.

```
namespace Snake
{
    3 referencias
    public partial class Form1 : Form
    {
        // Objeto principal de la serpiente
        Serpiente snake;
```

Guarda la siguiente dirección seleccionada por el jugador.

```
// Guarda la siguiente dirección elegida
Direccion siguienteDireccion;
```

Representa la comida que aparece en el tablero.

```
// Objeto comida
Comida comida;
```

Genera posiciones aleatorias para la comida.

```
// Generador de números aleatorios
Random random = new Random();
```

Define el tamaño de cada cuadro del tablero.

```
// Tamaño de cada cuadro del tablero
int tamañoCelda = 25;
```

Definen las dimensiones del tablero del juego.

```
// Cantidad de filas y columnas
int filas = 20;
int columnas = 20;
```

Almacena la puntuación actual del jugador.

```
// Puntaje del jugador  
int puntaje = 0;
```

Funcionalidad 2: Constructor principal

Activación de DoubleBuffered: Reduce el parpadeo gráfico durante el movimiento.

Activación de KeyPreview: Permite detectar teclas directamente desde el formulario.

```
1 referencia  
public Form1()  
{  
    InitializeComponent();  
  
    // Reduce parpadeos  
    this.DoubleBuffered = true;  
  
    // Permite detectar teclas en el formulario  
    this.KeyPreview = true;  
}
```

Asignación de eventos: Relaciona eventos con métodos específicos del juego.

```
// Eventos  
this.KeyDown += FrmSnake_KeyDown;  
timerJuego.Tick += timerJuego_Tick;  
btnReiniciar.Click += btnReiniciar_Click;  
btnSalir.Click += btnSalir_Click;
```

Inicio del juego: Inicializa todos los elementos necesarios para comenzar.

```
// Inicia el juego  
IniciarJuego();  
}
```

Funcionalidad 3: Inicialización del juego

Método IniciarJuego: Este método prepara completamente una nueva partida.

Creación de la serpiente

```
3 referencias  
private void IniciarJuego()  
{  
    snake = new Serpiente();  
}
```

Dirección inicial

```
// Dirección inicial  
siguienteDireccion = snake.DireccionActual;
```

Generación de comida

```
// Genera comida  
GenerarComida();
```

Reinicio del puntaje

```
// Reinicia puntaje  
puntaje = 0;  
  
lblPuntaje.Text = "Puntaje: 0";
```

Velocidad del juego: Cuando se aumenta más lento es y si se reduce más rápido es

```
// Velocidad del juego
timerJuego.Interval = 180;
```

Inicio del timer

```
// Inicia timer
timerJuego.Start();
}
```

Funcionalidad 4: Generación de comida

Método GenerarComida: Se encarga de la generación de la comida y que la comida nunca aparece encima de la serpiente.

Generación aleatoria

```
// Genera comida en posición libre
2 referencias
private void GenerarComida()
{
    bool posicionValida = false;

    int x = 0;
    int y = 0;

    while (!posicionValida)
    {
        x = random.Next(columnas);
        y = random.Next(filas);

        posicionValida = true;
    }
}
```

Validación de posición

```
// Verifica que la comida
// no aparezca sobre la serpiente
foreach (Posicion parte in snake.Cuerpo)
{
    if (parte.X == x && parte.Y == y)
    {
        posicionValida = false;
        break;
    }
}
```

Creación de comida

```
comida = new Comida(x, y);
}
```



Funcionalidad 5: Dibujo del juego

Método OnPaint: Todo el escenario visual se dibuja dinámicamente en cada actualización.

```
// Dibuja el juego
0 referencias
protected override void OnPaint(PaintEventArgs e)
{
    base.OnPaint(e);

    Graphics g = e.Graphics;
```

Dibujar tablero

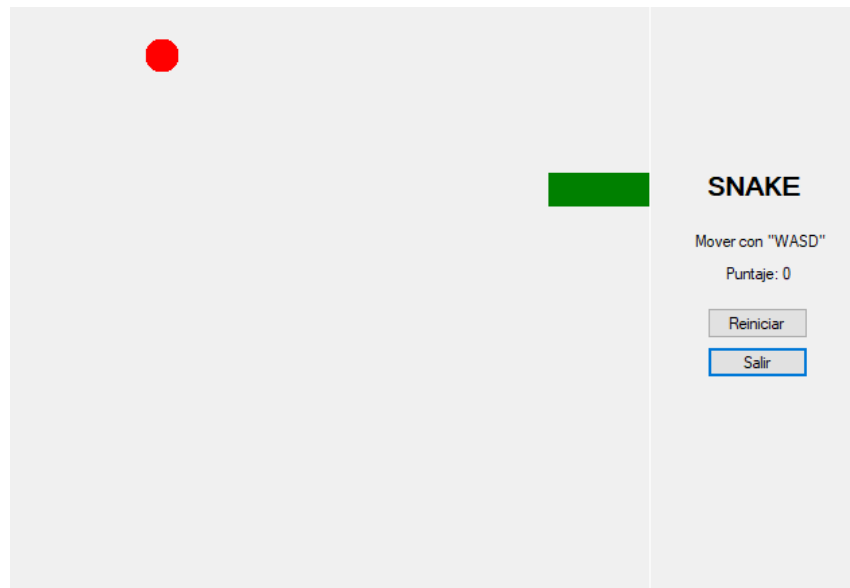
```
// Dibuja borde del tablero
g.DrawRectangle(
    Pens.White,
    0,
    0,
    columnas * tamañoCelda,
    filas * tamañoCelda);
```

Dibujar serpiente

```
// Dibuja serpiente
foreach (Posicion parte in snake.Cuerpo)
{
    g.FillRectangle(
        Brushes.Green,
        parte.X * tamañoCelda,
        parte.Y * tamañoCelda,
        tamañoCelda,
        tamañoCelda);
}
```

Dibujar comida

```
// Dibuja comida
g.FillEllipse(
    Brushes.Red,
    comida.Ubicacion.X * tamañoCelda,
    comida.Ubicacion.Y * tamañoCelda,
    tamañoCelda,
    tamañoCelda);
}
```

Funcionalidad 6: Control mediante teclado

Evento FrmSnake_KeyDown: El jugador controla la serpiente usando las teclas WASD.

Movimiento arriba

```
1 referencia
private void FrmSnake_KeyDown(object sender, KeyEventArgs e)
{
    // Arriba
    if (e.KeyCode == Keys.W &&
        snake.DireccionActual != Direccion.Abajo)
    {
        siguienteDireccion = Direccion.Arriba;
    }
}
```

Movimiento abajo

```
if (e.KeyCode == Keys.S &&
    snake.DireccionActual != Direccion.Arriba)
{
    siguienteDireccion = Direccion.Abajo;
}
```

Movimiento izquierda

```
if (e.KeyCode == Keys.A &&
    snake.DireccionActual != Direccion.Derecha)
{
    siguienteDireccion = Direccion.Izquierda;
}
```

Movimiento derecha

```
if (e.KeyCode == Keys.D &&
    snake.DireccionActual != Direccion.Izquierda)
{
    siguienteDireccion = Direccion.Derecha;
}
```

Funcionalidad 7: Enumeración Direccion

Organiza las direcciones posibles del movimiento.

Direcciones disponibles

- Arriba
- Abajo
- Izquierda
- Derecha

```
15 referencias
public enum Direccion
{
    Arriba,
    Abajo,
    Izquierda,
    Derecha
}
```

Funcionalidad 8: Clase Posicion

Representa coordenadas dentro del tablero.

Propiedades:

- X
- Y

```
15 referencias
public class Posicion
{
    13 referencias
    public int X { get; set; }
    13 referencias
    public int Y { get; set; }

    5 referencias
    public Posicion(int x, int y)
    {
        X = x;
        Y = y;
    }
}
```

Funcionalidad 9: Clase Comida

Ubicación de comida: Almacena la posición actual de la comida.

```
3 referencias
public class Comida
{
    5 referencias
    public Posicion Ubicacion { get; set; }

    1 referencia
    public Comida(int x, int y)
    {
        Ubicacion = new Posicion(x, y);
    }
}
```

Funcionalidad 10: Clase Serpiente

Lista del cuerpo: Almacena cada segmento de la serpiente.

```
3 referencias
public class Serpiente
{
    // Lista del cuerpo
    17 referencias
    public List<Posicion> Cuerpo { get; set; }
}
```

Dirección actual

```
8 referencias
public Direccion DireccionActual { get; set; }
```



Construcción inicial: Define el tamaño y posición inicial de la serpiente.

```
1 referencia
public Serpiente()
{
    Cuerpo = new List<Posicion>();

    // Tamaño inicial
    Cuerpo.Add(new Posicion(5, 5));
    Cuerpo.Add(new Posicion(4, 5));
    Cuerpo.Add(new Posicion(3, 5));

    DireccionActual = Direccion.Derecha;
}
```

Funcionalidad 11: Movimiento de la serpiente

Método Mover: La serpiente avanza agregando una nueva cabeza y eliminando la cola.

Creación de nueva cabeza

```
1 referencia
public void Mover(bool crecer)
{
    Posicion nuevaCabeza =
        new Posicion(Cuerpo[0].X, Cuerpo[0].Y);
}
```

Cambio de posición

```
switch (DireccionActual)
{
    case Direccion.Arriba:
        nuevaCabeza.Y--;
        break;

    case Direccion.Abajo:
        nuevaCabeza.Y++;
        break;

    case Direccion.Izquierda:
        nuevaCabeza.X--;
        break;

    case Direccion.Derecha:
        nuevaCabeza.X++;
        break;
}
```

Agregar cabeza

```
// Agrega nueva cabeza
Cuerpo.Insert(0, nuevaCabeza);
```

Eliminar cola

```
// Si no come, elimina cola
if (!crecer)
{
    Cuerpo.RemoveAt(Cuerpo.Count - 1);
}
```

Funcionalidad 12: Detección de colisiones

Método ChocaConCuerpo: Detecta si la serpiente choca consigo misma.

Comparación de posiciones

```
1 referencia
public bool ChocaConCuerpo()
{
    Posicion cabeza = Cuerpo[0];

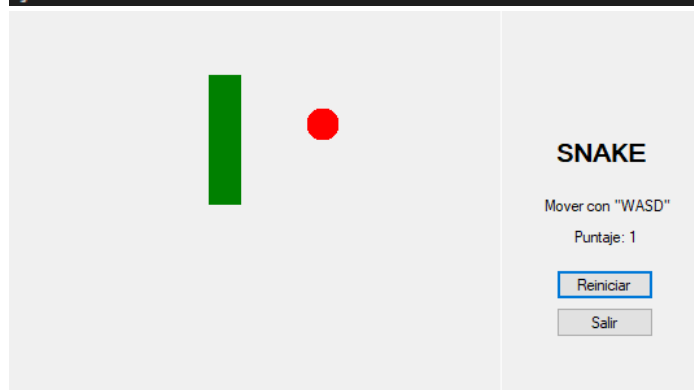
    for (int i = 1; i < Cuerpo.Count; i++)
    {
        if (cabeza.X == Cuerpo[i].X &&
            cabeza.Y == Cuerpo[i].Y)
        {
            return true;
        }
    }

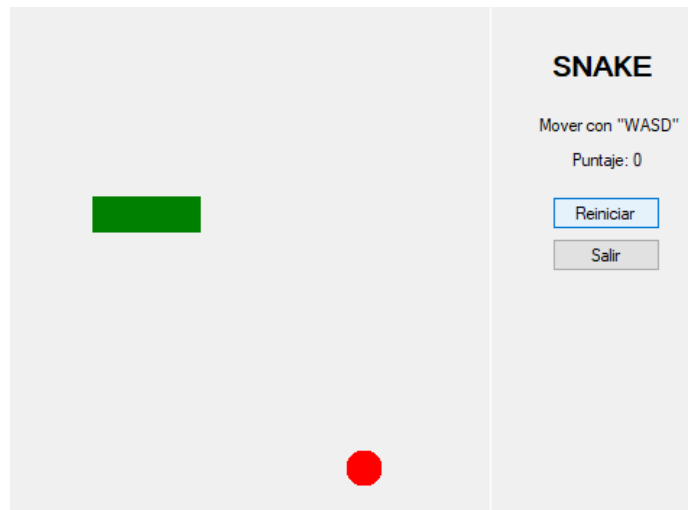
    return false;
}
```

**Funcionalidad 13: Reinicio del juego**

Evento btnReiniciar_Click: Permite comenzar una nueva partida.

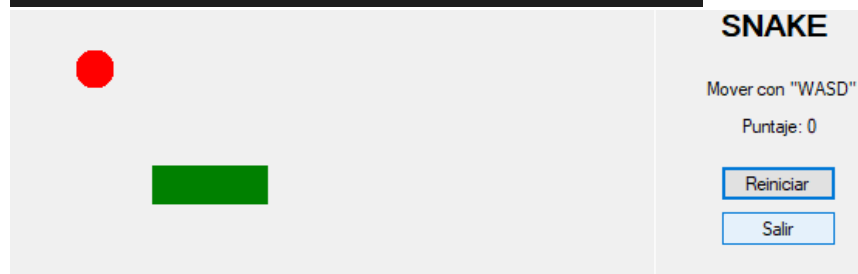
```
private void btnReiniciar_Click(object sender, EventArgs e)
{
    IniciarJuego();
}
```



**Funcionalidad 14: Salida del sistema**

Evento btnSalir_Click: Cierra completamente la aplicación.

```
2 referencias
private void btnSalir_Click(object sender, EventArgs e)
{
    Application.Exit();
}
```

**Funcionalidad 15: Lógica principal del juego**

Evento timerJuego_Tick: Este evento controla todo el funcionamiento del videojuego:

- Movimiento.
- Colisiones.
- Puntaje.
- Crecimiento.
- Reinicio.
- Actualización gráfica.
-

Obtención de cabeza

```
2 referencias
private void timerJuego_Tick(object sender, EventArgs e)
{
    Posicion cabezaActual = snake.Cuerpo[0];
}
```

Detección de comida

```
bool comer =
    cabezaActual.X == comida.Ubicacion.X &&
    cabezaActual.Y == comida.Ubicacion.Y;
```



Actualización de dirección

```
snake.DireccionActual = siguienteDireccion;
```

Movimiento

```
snake.Mover(comer);  
Posicion cabeza = snake.Cuerpo[0];
```

Actualización del puntaje

```
if (comer)  
{  
    puntaje++;  
}
```

Actualizar etiqueta

```
lblPuntaje.Text = "Puntaje: " + puntaje;
```

Generar nueva comida

```
    GenerarComida();  
}
```

Detección de bordes

```
bool chocaBorde =  
    cabeza.X < 0 ||  
    cabeza.Y < 0 ||  
    cabeza.X >= columnas ||  
    cabeza.Y >= filas;
```

Colisión con cuerpo

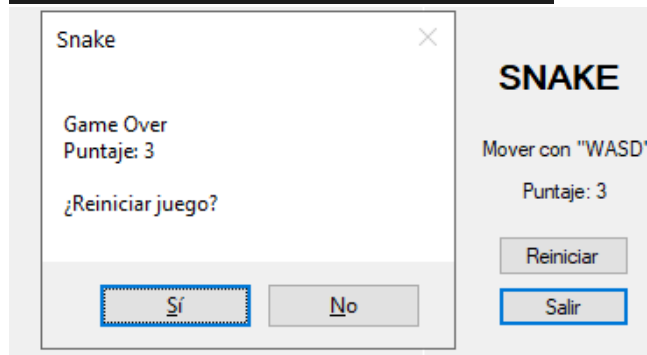
```
bool chocaCuerpo = snake.ChocaConCuerpo();
```

Fin del juego

```
if (chocaBorde || chocaCuerpo)  
{  
    timerJuego.Stop();  
}
```

Reinicio o salida

```
DialogResult resultado =  
    MessageBox.Show(  
        "Game Over\nPuntaje: " + puntaje +  
        "\n\n¿Reiniciar juego?",  
        "Snake",  
        MessageBoxButtons.YesNo);  
  
if (resultado == DialogResult.Yes)  
{  
    IniciarJuego();  
}  
else  
{  
    Application.Exit();  
}  
  
return;  
}
```



Redibujar pantalla

```
// Redibujar pantalla  
Invalidate();  
}
```



IV. ANALISIS E INTERPRETACION DE RESULTADOS

¿Qué indican los resultados?

- El juego se ejecuta de forma adecuada en tiempo real.
- La serpiente responde adecuadamente a las pulsaciones del teclado.
- Las colisiones y los resultados se controlan correctamente

¿Que se ha encontrado?

- Timer permite que simule un movimiento continuo.
- Graphics permite hacer el dibujo dinámico.
- Las listas permiten gestionar de manera sencilla el cuerpo de la serpiente.
- La programación orientada a objetos permite tener un código mejor organizado.

CONCLUSIONES

- Se produjo de forma satisfactoria el videojuego Snake a partir del lenguaje C# y Windows Forms.
- El sistema logró controlar correctamente el movimiento y el crecimiento de la serpiente.
- La implementación de colisiones permitía gestionar correctamente el fin del videojuego.
- El uso de eventos de teclado permitía mejorar la interacción con el usuario.
- Graphics permitía representar visualmente todos los elementos del videojuego.
- La programación orientada a objetos mejoró la organización y el mantenimiento del mantenimiento del proyecto.
- La utilización de listas dinámicas permitía gestionar de forma eficiente el cuerpo de la serpiente.
- Este proyecto sirve para reforzar los conocimientos sobre lógica de videojuegos, eventos y programación gráfica.

RECOMENDACIONES

- Implementar diferentes niveles de dificultad.
- Incluir el sistema de records.
- Añadir la pausa del juego.



WEBGRAFIA

BillWagner. (s. f.). Operadores y expresiones: enumerar todos los operadores y expresiones - C# reference. Microsoft Learn. Recuperado de <https://learn.microsoft.com/es-es/dotnet/csharp/language-reference/operators/>

Arreglos en C#. (2014). C# Paso a Paso. Recuperado de <https://csharp-facilito.blogspot.com/2013/07/arreglos-en-c-sharp.html>

Turrado, J. (2022). Introducción rápida a LINQ con C#: manejar información en memoria nunca fue tan sencillo - campusMVP.es. campusMVP.es. Recuperado de https://www.campusmvp.es/recursos/post/introduccion-rapida-a-linq-con-c-sharp.aspx?srsId=AfmBOoqx5_WZg2mtHdV0_YFJiBZsZ5dj_t3Us87MablaeW1k-0upk2xK